EXAM SEATING ARRANGEMENT

A Branch-Aware Seat Allocator, Built with CSP Backtracking

AI Agent | CSP Backtracking | Flask Web App

CSE ECE MECH CIVIL EEE

BTech AI — College Project

PROBLEM STATEMENT

CO-1 ALIGNED

The task is to build a web tool that assigns each student a unique seat in a rows×cols exam grid, **keeping any two students from the same branch out of adjacent seats** so neighbours can't easily compare answers.



Name + branch as input

Each student is added through a form field: name and a branch dropdown (CSE / ECE / MECH / CIVIL / EEE).



No same-branch neighbours

A seat's up, down, left, and right neighbours must never hold a student from the same branch.



One student per seat

Seats are filled one at a time during search; an already-filled seat is never reassigned.



Grid capacity respected

If the student count exceeds rows × cols, the solver reports no valid arrangement.

PEAS MODEL

CO-1 ALIGNED

P PERFORMANCE

- A complete grid with no same-branch seats touching
- Zero seats double-booked
- Clear message when no layout is possible

E ENVIRONMENT

- The rows × cols seat grid rendered in-browser
- The student roster, each tagged with a branch
- A single exam session (no multi-hall support yet)

A ACTUATORS

- Renders the grid as colour-coded tiles per branch
- Displays each student's name on their tile
- Shows an error if no arrangement was found

S SENSORS

- Rows, cols, student name & branch from the form
- The “Solve” click that fires the POST request

ALGORITHM: CSP

CO-2 & CO-3

VARIABLES

Students waiting for a seat, sorted by branch before search starts.

DOMAIN

Every empty (row, col) cell in the rows × cols grid.

CONSTRAINTS

1 student/seat · no same-branch neighbour (↑↓←→) · students ≤ rows×cols.

WHY CSP?

Seating is an assignment problem: pick a seat (value) for every student (variable) under constraints. CSP models that directly — there's no path between two points to search for.

WHY NOT DFS / BFS / UCS / A* / GREEDY?

Those algorithms traverse a graph of states toward a goal node. There's no such path here — just a labelling task — so backtracking search with constraint-checking and undo fits the problem instead.

THE REAL CONSTRAINT



No same-branch seat directly above, below, left, or right.

IMPLEMENTATION

CO-3

app.py — solve_seating()

```
def is_valid(r, c, student):
    branch = student['branch']
    for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
        nr, nc = r+dr, c+dc
        if 0 <= nr < rows and 0 <= nc < cols:
            if seats[nr][nc] and \
               seats[nr][nc]['branch'] == branch:
                return False
    return True

def backtrack(remaining):
    if not remaining: return True
    for r in range(rows):
      for c in range(cols):
        if seats[r][c] is None:
          for i, student in enumerate(remaining):
            if is_valid(r, c, student):
              seats[r][c] = student
              if backtrack(remaining[:i]+remaining[i+1:]):
                  return True
              seats[r][c] = None
          return False
    return True
```

is_valid()

Rejects a seat if any of its 4 orthogonal neighbours already holds a student from the same branch.

backtrack()

Fills one seat at a time; if a path leads nowhere, it undoes the choice (seats[r][c] = None) and tries again.

Result

Returns the filled grid on success, or None if no valid layout exists for the given roster.

CO ATTAINMENTS

CO-1

AI Agent & PEAS

Modelled the Flask tool as an agent: sensors read the roster + grid size from the form; actuators render the colour-coded grid.

CO-2

Search Algorithms

Justified CSP backtracking over DFS, BFS, UCS, and A* — seating is an assignment task, not a path through a state graph.

CO-3

Constraint Satisfaction

Variables = students, domain = empty seats, constraints = no same-branch neighbours + one student per seat + capacity.

CO-4

Utility & Decision

backtrack() tries a student per seat, undoes a bad choice, and returns success or failure straight to the UI.

CO-5

Probabilistic Inference

Future extension: predict likely no-shows from attendance history to free up seats automatically — not yet built.

CO-6

Hybrid Architecture

Combines the CSP solver with a live Flask + JS interface so the resulting grid can be checked by eye, seat by seat.

THANK YOU

Examination Seating Arrangement | AI Course Project

CSE ECE MECH CIVIL EEE